

# Logs and Metrics

---

**Arturo Silvelo**

Try New Roads

# Logs

# Logs

---

El comando `docker logs` permite visualizar la salida generada por los procesos que se ejecutan dentro de un contenedor. Por defecto, muestra todo lo que la aplicación escribe en los flujos estándar de salida y error (STDOUT y STDERR), tal como lo veríamos si ejecutáramos el proceso en una terminal. Sin embargo, la utilidad de este comando depende de cómo la aplicación gestione sus propios logs. Si la aplicación escribe sus registros en archivos internos o si se utiliza un driver de logs externo, es posible que `docker logs` no muestre información relevante.

## Logging drivers

---

Docker ofrece varios mecanismos para gestionar y recolectar los logs de los contenedores y servicios, conocidos como `logging drivers` o controladores de logs. Estos controladores permiten enviar los registros a distintos destinos.

- **json-file**: Es el driver por defecto. Guarda los logs en archivos JSON en el host.
- **syslog**: Envía los logs al sistema syslog del host o a un servidor remoto.
- **fluentd**: Permite enviar los logs a un agregador Fluentd.
- **awslogs**: Envía los logs a Amazon CloudWatch Logs.
- **gelf**: Envía los logs a sistemas compatibles con Graylog Extended Log Format.
- **none**: Desactiva el registro de logs para el contenedor.

## Configuración

---

Puedes configurar el comportamiento de los logs de manera global para todos los contenedores editando el archivo `/etc/docker/daemon.json`.

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3",
    "labels": "production_status",
    "env": "os,customer"
  }
}
```

- Ejecución (Sin logs)

```
docker run -it --rm --init --log-driver none --name app ghcr.io/trynewroads/docker-course-advance
```

- Verificación

```
$docker logs app  
Error response from daemon: configured logging driver does not support reading
```

- Ejecución (Json-file)

```
docker run -it --rm --init --log-opt labels=environment --log-opt env=os --label environment=production -e os=ubuntu --name app ghcr.io/trynewroads/docker-course-advance
```

- Verificación

```
sudo cat /var/lib/docker/containers/<container_id>/<container_id>-json.log | jq
```

# GELF

---



## GELF

---

El driver **GELF** (Graylog Extended Log Format) permite enviar los logs de los contenedores Docker a sistemas de gestión de logs compatibles con Graylog, como Graylog, Logstash o Fluentd. GELF es útil para centralizar, analizar y visualizar logs de múltiples contenedores en una sola plataforma.

- Permite enviar logs en formato estructurado (JSON) a través de UDP o TCP.
- Facilita la integración con herramientas de monitoreo y análisis.
- Soporta la inclusión de metadatos como etiquetas y variables de entorno.

| Opción              | Requerido | Descripción                                     | Ejemplo                                                      |
|---------------------|-----------|-------------------------------------------------|--------------------------------------------------------------|
| <b>gelf-address</b> | Sí        | Dirección del servidor GELF (UDP/TCP y puerto). | <code>--log-opt gelf-address=udp://192.168.0.42:12201</code> |
| <b>env</b>          | No        | Lista de variables de entorno a incluir.        | <code>--log-opt env=os,customer</code>                       |
| <b>labels</b>       | No        | Lista de labels a incluir como campos extra.    | <code>--log-opt labels=production_status,geo</code>          |

```
name: log
# For DataNode setup, graylog starts with a preflight UI, this is a change from just using OpenSearch/Elasticsearch.
# Please take a look at the README at the top of this repo or the regular docs for more info.

services:
  # MongoDB: https://hub.docker.com/_/mongo/
  mongodb:
    image: "mongo:7.0"
    restart: "on-failure"
    networks:
      - log
    volumes:
      - "mongodb_data:/data/db"
      - "mongodb_config:/data/configdb"

  # For DataNode setup, graylog starts with a preflight UI, this is a change from just using OpenSearch/Elasticsearch.
  # Please take a look at the README at the top of this repo or the regular docs for more info.
  # Graylog Data Node: https://hub.docker.com/r/graylog/graylog-datanode

  # ⚠️ Make sure this is set on the host before starting:
  # echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
  # sudo sysctl -p
  datanode:
    image: "${DATANODE_IMAGE:-graylog/graylog-datanode:6.3}"
    hostname: "datanode"
    environment:
      GRAYLOG_DATANODE_NODE_ID_FILE: "/var/lib/graylog-datanode/node-id"
      # GRAYLOG_DATANODE_PASSWORD_SECRET and GRAYLOG_PASSWORD_SECRET MUST be the same value
      GRAYLOG_DATANODE_PASSWORD_SECRET: "${GRAYLOG_PASSWORD_SECRET:?Please configure GRAYLOG_PASSWORD_SECRET in the .env file}"
      GRAYLOG_DATANODE_MONGODB_URI: "mongodb://mongodb:27017/graylog"
    ulimits:
      memlock:
        hard: -1
        soft: -1
      nofile:
        soft: 65536
        hard: 65536
    ports:
      - "8999:8999/tcp" # DataNode API
      - "9200:9200/tcp"
      - "9300:9300/tcp"
    networks:
      - log
    volumes:
      - "graylog-datanode:/var/lib/graylog-datanode"
    restart: "on-failure"

  # Graylog: https://hub.docker.com/r/graylog/graylog-enterprise
  graylog:
    hostname: "server"
    image: "${GRAYLOG_IMAGE:-graylog/graylog:6.3}"
    depends_on:
      mongodb:
        condition: "service_started"
      datanode:
        condition: "service_started"
    entrypoint: "/usr/bin/tini -- /docker-entrypoint.sh"
    environment:
      GRAYLOG_NODE_ID_FILE: "/usr/share/graylog/data/data/node-id"
      # GRAYLOG_DATANODE_PASSWORD_SECRET and GRAYLOG_PASSWORD_SECRET MUST be the same value
      GRAYLOG_PASSWORD_SECRET: "${GRAYLOG_PASSWORD_SECRET:?Please configure GRAYLOG_PASSWORD_SECRET in the .env file}"
      GRAYLOG_ROOT_PASSWORD_SHA2: "${GRAYLOG_ROOT_PASSWORD_SHA2:?Please configure GRAYLOG_ROOT_PASSWORD_SHA2 in the .env file}"
      GRAYLOG_HTTP_BIND_ADDRESS: "0.0.0.0:9000"
      GRAYLOG_HTTP_EXTERNAL_URI: "http://localhost:9000/"
      GRAYLOG_MONGODB_URI: "mongodb://mongodb:27017/graylog"
    ports:
      - "5044:5044/tcp" # Beats
      - "5140:5140/udp" # Syslog
      - "5140:5140/tcp" # Syslog
      - "5555:5555/tcp" # RAW TCP
      - "5555:5555/udp" # RAW UDP
      - "9000:9000/tcp" # Server API
      - "12201:12201/tcp" # GELF TCP
      - "12201:12201/udp" # GELF UDP
      - "13000:13000/tcp" # Custom TCP port
      - "13000:13000/udp" # Custom UDP port
      - "13301:13301/tcp" # Forwarder data
      - "13302:13302/tcp" # Forwarder config
    networks:
      - log
    volumes:
      - "graylog_data:/usr/share/graylog/data/data"
    restart: "on-failure"

networks:
  log:
    driver: "bridge"
    name: log-network

volumes:
  mongodb_data:
  mongodb_config:
  graylog-datanode:
  graylog_data:
```

## • Ejecución

```
docker compose up -d
```

## • Verificación

```
docker logs log-graylog-1
Initial configuration is accessible at 0.0.0.0:9000, with username 'admin' and password 'QadtbPdtsu'.
Try clicking on http://admin:QadtbPdtsu@0.0.0.0:9000
```

## • Generar CA

## • Iniciar Sesión: admin/admin1234

## • Configurar System -> Input: GELF UDP

## • Configurar Stream

# Metrics

# Metrics

---